Informed Search

limck@um.edu.my

Contents

1 Informed Search Algorithms **1**
1.1 Greedy Best-First Search (GBFS) 3
1.2 A* Search Algorithm 5
1.3 Weighted A* Search 7
1.4 Beam Search 9
1.5 Iterative-Deepening A* Search (IDA*) . 9
2 Heuristic Functions **11**
2.1 Understanding Heuristic Functions . . . 11
2.2 Branching Factor 11
2.3 Defining Heuristic With Relaxed Problems 13

1 Informed Search Algorithms

If you want to find a path from Kuala Lumpur to Georgetown, Penang, you might consider exploring Sekinchan or Tanjung Malim (Figure 1). However, would you also consider exploring Seremban?

If you have an idea where all these cities are on a Malaysia map, definitely you will not consider Seremban. The reason is simple, it is not leading us to the goal we want to reach. The feature of a state (in this example, the straight line distance [1] of

[1] Smaller straight line distance does not means smaller real distance, but it serve as a hints

Seremban from Georgetown on the map) give us hints about the relation of state and the goal.

Figure 1: A Malaysia map showing Kuala Lumpur, Georgetown, Sekinchan, Tanjung Malim and Seremban.

We can create mathematical functions based on this type of information, which we refer to as heuristic functions , $h(n)$. Informed Search utilizes heuristic information to enhance the efficiency of searches.

heuristic functions

informed search

In this section, we will discuss a few algorithms that make use of heuristic information to search for a goal. The example we are using is based on the route finding problem in Romania (Figure 2). We want to find a path from Arad to Bucharest.

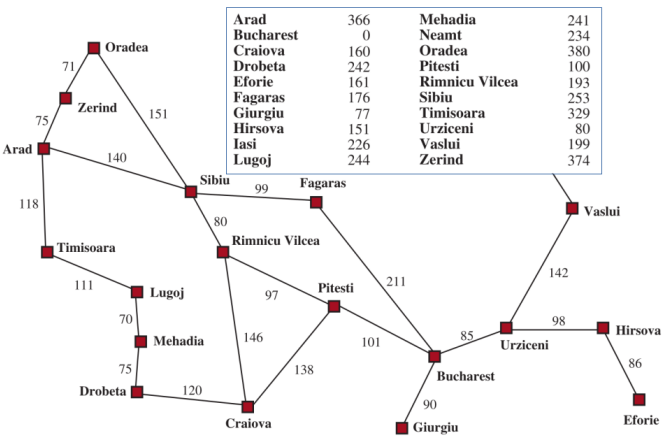


Figure 2: Romania map with straight line distances from cities to Bucharest.

1.1 Greedy Best-First Search (GBFS)

Greedy best-first search is a type of greedy algorithm. A greedy algorithm is a problem-solving strategy that makes the most attractive, locally optimal option at each individual step with the hope of finding a solution. However, greed may lead to worse outcomes.

greedy best-first search

greedy algorithm

It works quite similar to the Uniform-Cost Search algorithm which uses an evaluation function to determine how a newly explored node being added to the priority queue, or the frontier. However, for the case of Uniform-Cost Search, the evaluation function, $f(n) = g(n)$, where $g(n)$ is the actual cost for state n (e.g. how far one has been traveled to reach city n in the route finding problem). For the case of GBFS, the evaluation function is $f(n) = h(n)$, where $h(n)$ is the heuristic function (e.g. straight line distance between city n and the goal).

evaluation function

Algorithm 1: Greedy Best-First Search Algorithm

```
Function GREEDY-BEST-FIRST-SEARCH(problem, h)
    node ← Node(problem.INITIAL)
    frontier ← a Priority Queue ordered by h, with node
    reached ← {problem.INITIAL}          // Track visited states
    while not Is-Empty(frontier) do
        node ← Pop(frontier)             // Pop lowest h(n)
        if problem.IS-GOAL(node.STATE) then
            return node
        foreach child in Expand(problem,node) do
            s ← child.STATE
            if s is not in reached then
                add s to reached
                add child to frontier
    return failure
```

GBFS works as follow (Algorithm 1): It starts with initial state. While at a state, it continues to explore neighbour states, and add neighbour states to the frontier list ordered by lowest

$f(n)$ values. The algorithm stops when it reaches the goal.

Some properties of GBFS:

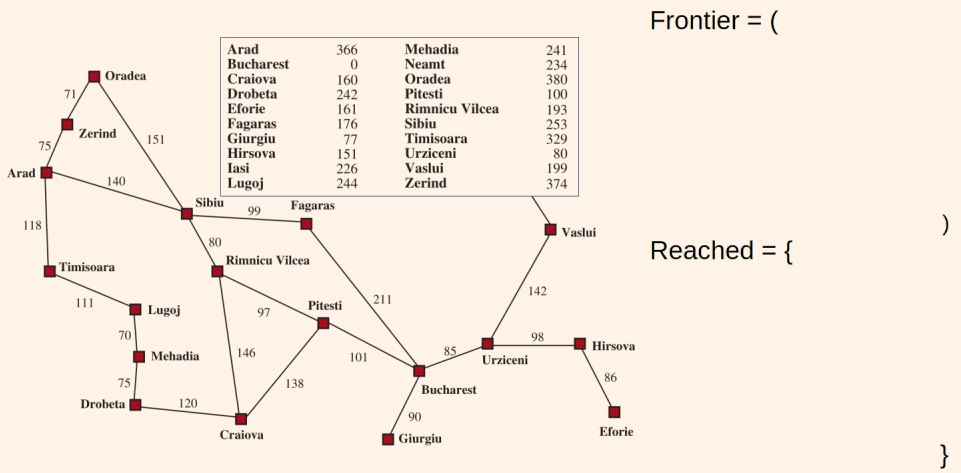
- Complete? No – can get stuck in loops.
- Time complexity? $O(b^m)$, but a good heuristic can give dramatic improvement.
- Space complexity? $O(b^m)$, it keeps all nodes in memory
- Optimal? No, as you can see in the example.

Note that the even the time and space complexity of GBFS for the worst case scenario are similar to Uniform-Cost Search, we still consider it is a faster algorithm because it is more "directional focus". If the cheapest solution is not a main concern, GBFS is a good option. Study Example 1 and compare the result of Uniform-Cost Search to find out more.

time and space complexity

Example 1: Greedy Best-First Search Algorithm

Using GBFS, find the path from Arad to Bucharest. Is the path it finds a shortest path?



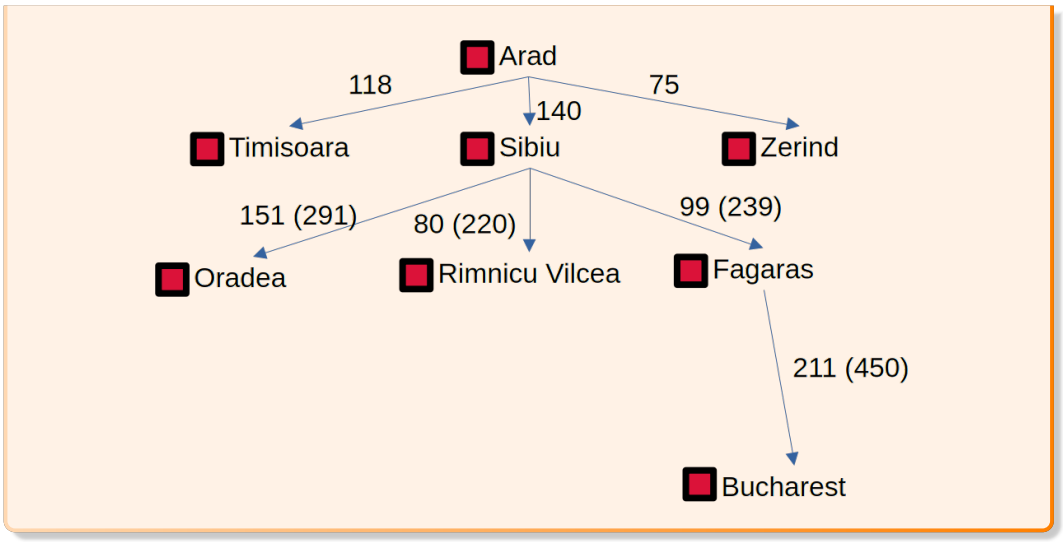
Frontier = (

)

Reached = {

}

Solving the problem, you should find that the GBFS algorithm finds a tree as follow. The distance of a city from Arad is written in the parentheses next to edges.



1.2 A* Search Algorithm

The main idea of A* search algorithm is: pick the node which looks most promising to explore, by evaluate each node with: A* search

$$f(n) = g(n) + h(n) \tag{1}$$

where $f(n)$ is the estimated total cost from root to goal through n; $g(n)$ is the actual cost from root to reach n (we have seen this in Uniform-Cost Search); and $h(n)$ is the estimated total cost from n to goal (we have seen this in GBFS).

After the evaluation, A* always pick node with the lowest estimated cost to explore.

Like the GBFS and other algorithm, it also maintain a frontier list (priority queue) and a reached set. Starting from the initial state, it continue to discover neighbour states, and put the one with lowest $f(n)$ at the top of frontier. The algorithm ends when the goal is removed from the frontier (refer Algorithm 2).

Some properties of A* search:

- Complete? Yes.
- Time complexity? Exponential, $O(b^d)$
- Space complexity? Exponential, $O(b^d)$
- Optimal? Yes, if the heuristic is admissible – heuristic estimations are not more than actual cost [2].

[2] For example, the straight line distance from city n to Bucharest is always less than or

Algorithm 2: A* Search Algorithm

```
Function A*-SEARCH(problem, h)
    node ← Node(problem.INITIAL, PATH_COST=0)
    frontier ← a Priority Queue ordered by f, with node
    reached ← {problem.INITIAL : 0}
    while not Is-Empty(frontier) do
        node ← Pop(frontier)  // Pop node with lowest f
        if problem.IS-GOAL(node.STATE) then
            return node
        foreach child in Expand(problem, node) do
            s ← child.STATE
            g ← child.PATH_COST     // Actual cost to s
            if s is not in reached or g < reached[s] then
                reached[s] ← g
                f ← g + h(s)     // f is the combine cost
                add child to frontier with priority f
    return failure
```

A* algorithm is complete, but suffer from a major problem - high space complexity. With a branching factor of 3, at level 15, the nodes generated can grow to $3^{15} = 1,073,741,824$. Do you think it is a good idea to implement A* search for route planning software without modification? Refer Figure 3.

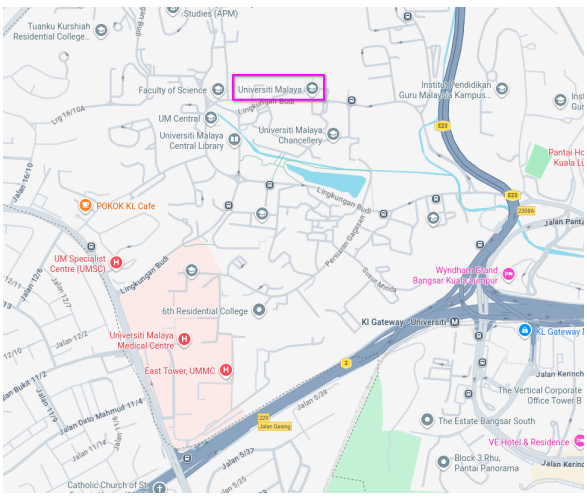
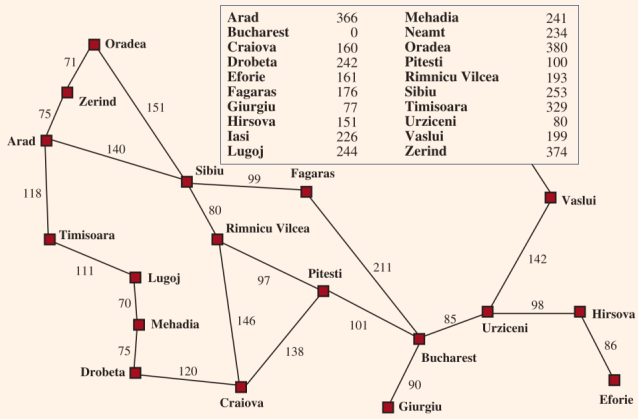


Figure 3: Part of UM and it's surrounding.

equal to actual road distance, $h(n) \leq Real(n)$.

Example 2: A* Search Algorithm

Using A*, find the path from Arad to Bucharest. Is the path it finds a shortest path?



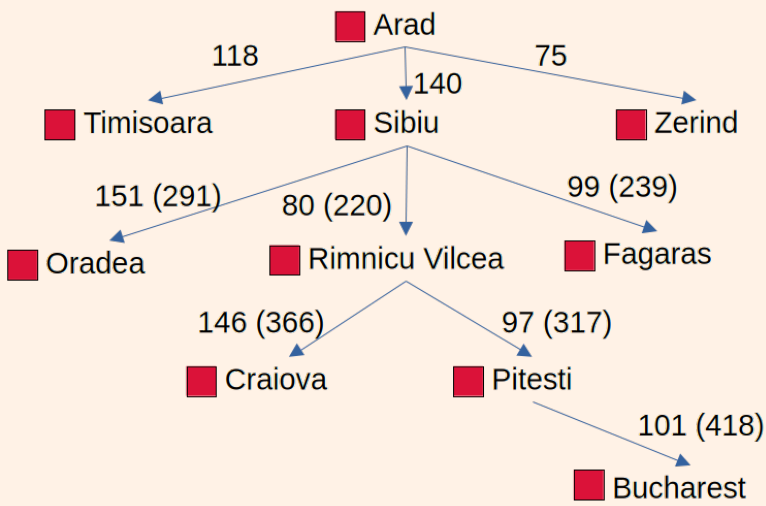
Frontier = (

)

Reached = {

}

Solving the problem, you should find that the A* algorithm finds a tree as follow. Is there any difference between the results of Uniform-Cost Search, GBFS and A*?



1.3 Weighted A* Search

Due to the high space complexity of the A* algorithm, there have been many efforts to improve it, for example Weighted A* Search, Beam Search and Iterative-Deepening A* Search.

Weighted A* Search makes the algorithm to be a bit "greedy" compared to standard A*, but not as greedy as GBFS. It allows

Weighted A* search

A* to use inadmissible heuristic, i.e. it accept:

$$h(n) \geq \textsc{Real}(n) \tag{2}$$

But there is a drawback, the solutions may be suboptimal, but satisficing (which means, good enough). satisficing

Assume that there is a relation between a straight line distance and a real road distance between two cities. This relation can be written as a multiplier. We can make use of this concept and assign a weight to A* search:

$$f(n) = g(n) + W \times h(n) \tag{3}$$

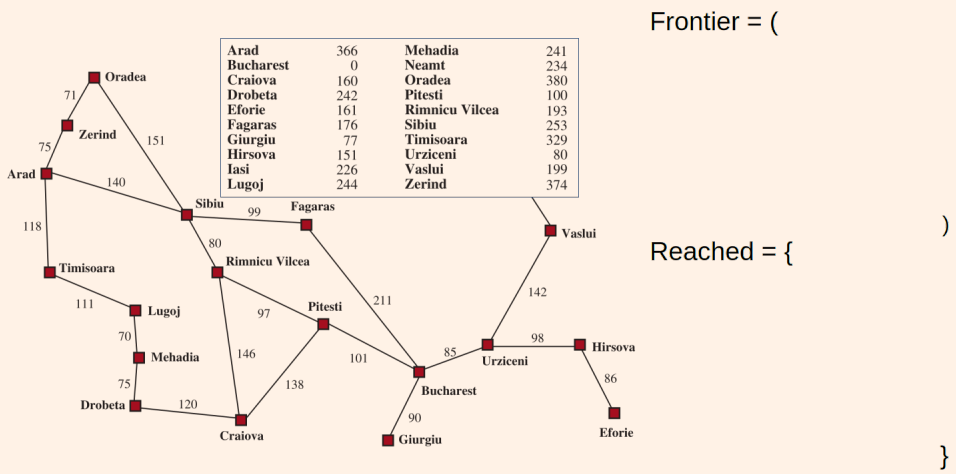
where $W > 1$.

In general, if optimal cost of a problem is $C*$, weighted A* finds a solution in $[C*, WC*]$, but usually closer to $C*$.

To modify A* algorithm to Weighted A*, just change line 14 in Algorithm 2 to Equation 3.

Example 3: Weighted A* Search Algorithm

Using Weighted A* with $W = 1.5$, find the path from Arad to Bucharest. Is the path it finds a shortest path?



Frontier = (

)

Reached = {

}

Do we find optimal solution with $W = 1.5$? What if we use $W = 1.001$?

1.4 Beam Search

Beam search takes a very simple approach to reduce the use of memory – limit the size of frontier to the best k nodes with best f-score, and returns the goal as soon as it is generated. This makes the algorithm work faster and requires less memory. However, it suffers from two issues: suboptimal and incomplete.

beam search

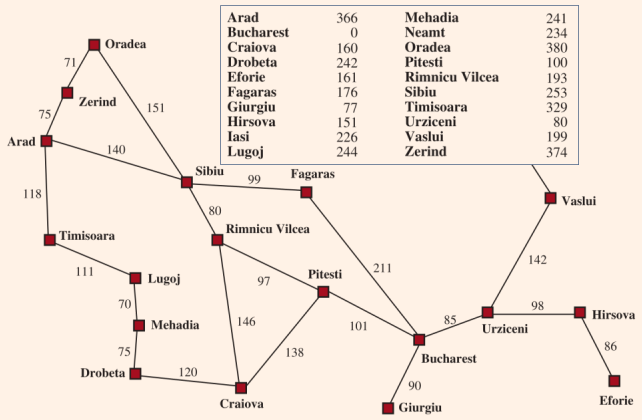
We can apply beam search to both GBFS and A* search. For Greedy Beam Search , $f(n) = h(n)$, and for A* Beam Search, $f(n) = g(n) + h(n)$.

greedy beam search

A* beam search

Example 4: Beam Search Algorithm

Using A* Beam Search with $k = 2$, find the path from Arad to Bucharest. Is the path it finds a shortest path?



Frontier = (

Reached = {

)

}

1.5 Iterative-Deepening A* Search (IDA*)

Iterative-Deepening A* Search combine the idea of Iterative-Deepening Search and A* Search. It does not need to keep all reached states in memory, but may need to revisit visited states.

iterative-deepening A* search

Recall that in standard Iterative Deepening Search, we explore all states at a depth of no more than l. If the goal is not found, the value of l is increased in the subsequent iteration.

In IDA*, the cutoff is not limited by l, but the value of f. At each iteration, the cutoff is the smallest f-cost of any node that exceeded the cutoff on previous iteration.

cutoff

The search works as follow:

1. It starts with root node and find its $f(n) = g(n) + h(n)$. This $f(n)$ is the cutoff, i.e. the maximum $f(n)$ allowed for further exploration.

2. Expand current node to find its children, along with their $f(n)$.

3. For this set of children, if their $f(n) >$ cutoff, prune them.

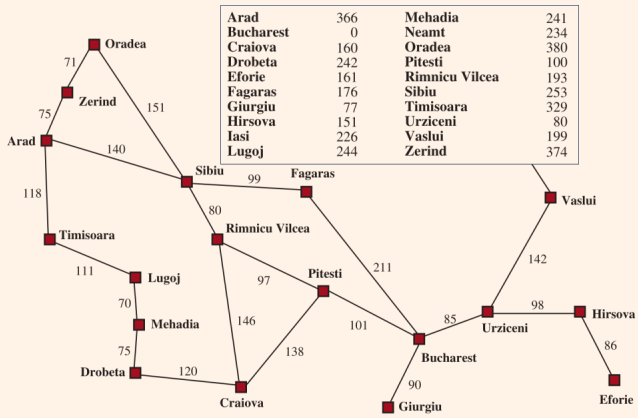
4. From the results not pruned, check if the goal is reached. If yes, return the node.

5. If no, update the cutoff to be the minimum pruned value from reached set, and continue from step 2.

Example 5: Iterative-Deepening A* Search

Using Iterative-Deepening A* Search, find the path from Arad to Bucharest. Record the value of cutoff of each iteration. Is the path it finds a shortest path?

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Frontier = (

)

Reached = {

}

2 Heuristic Functions

2.1 Understanding Heuristic Functions

"Straight line distance" is the heuristic we used in the Romania map problem. However:

1. Straight line distance is not the only possible heuristic. Other examples include distance from some landmark (e.g. Craiova, Fagaras).

2. Straight line distance is not applicable to some other problems, like solving puzzle, assembly complex objects, ...

Heuristic function is a function that estimate cost of the cheapest path from a state to the goal state. For a problem, we may have more than one way to define heuristic functions.

heuristic function

2.2 Branching Factor

The quality of a heuristic is measured by effective branching factor , $b*$ defined as follow:
If A* search generates N nodes for a problem, and find solution at level d:

effective branching factor

$$N + 1 = (b*)^0 + (b*)^1 + (b*)^2 + (b*)^3 + \cdots + (b*)^d \qquad (4)$$

There is no simple solution for this equation when d increases. Usually it is solved with numerical methods.

Example 6: Branching Factor Calculation

If 30 nodes is generated and the problem is solved in level 4:

$$30 + 1 = (2)^0 + (2)^1 + (2)^2 + (2)^3 + (2)^4$$

So, the branching factor is 2.00.

A good branching factor has a value close to 1.00. A small branching factor is better than a large one, because it reduces the time and space complexity.

However, when you have 2 similar ways to define heuristics, usually the one with higher heuristic values is better.

Example 7: Branching Factor of 8-Puzzle

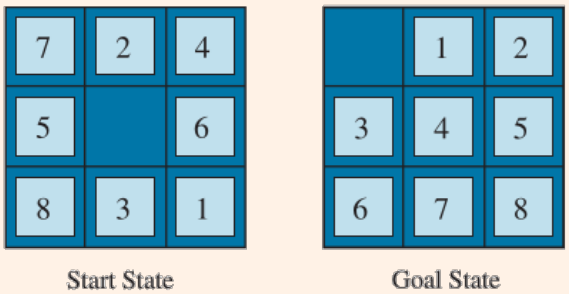


Figure 4: 8-puzzle

There are at least 2 ways to define heuristic for 8-puzzle.

1. h_1 : the number of misplaced tiles (exclude blank)
2. h_2 : sum of Manhattan distance of the tiles from their goal position. Generally, $h2 > h1$.

For the case of Start State in Fig. 4:

1. $h_1 = 8$, this is because all the 8 tiles are not on their goal position.
2. $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$. We need 3 slides to put '7' at the goal position, 1 slide to put '2' at the goal position. etc.

Fig. 5 compared the cost of searches required by BFS and A* with two different ways of defining heuristic. The 'cost' mentioned here refers to the nodes generated to complete the search. The effective branching factors are also compared. Obviously, A* has lower search cost and branching factor compared to BFS. But for A* itself, the version with larger heuristic value in general (i.e. h_2) has lower search cost and branching factor.

	Search Cost (nodes generated)			Effective Branching Factor		
d	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Figure 5: Data are averaged over 100 puzzles for each solution length d from 6 to 28.

2.3 Defining Heuristic With Relaxed Problems

There are many ways to define heuristic. One of it is based on relaxed problems . A relaxed problem is a problem with fewer restriction compared to the one to be solved. Technically, it can be formed by adding edges to the state-space graph of the problem. This is because an optimal solution for the original problem is also a solution of the relaxed problem.

relaxed problem

The straight line distance discussed earlier is an examples of relaxed problem. It let us travel in straight line, which is not possible in real case.

Both h_1 and h_2 Example 7 are defined with relaxed problem. h_1 assumes we can remove a tiles and place it directly to the goal position, whereas h_2 assumes there is no other tiles blocking the way of each tile to their goal.